Modellerin ve Tokenizerların Paylaşılması

Doğal Dil İşlemenin Merkezi: Hugging Face Hub, Model Yönetimi ve Kartları



İçindekiler ve Yol Haritası: Bölüm 4 Akışı



01. **Keşif ve Tüketim:**
Model depoları ve Auto* sınıfları.

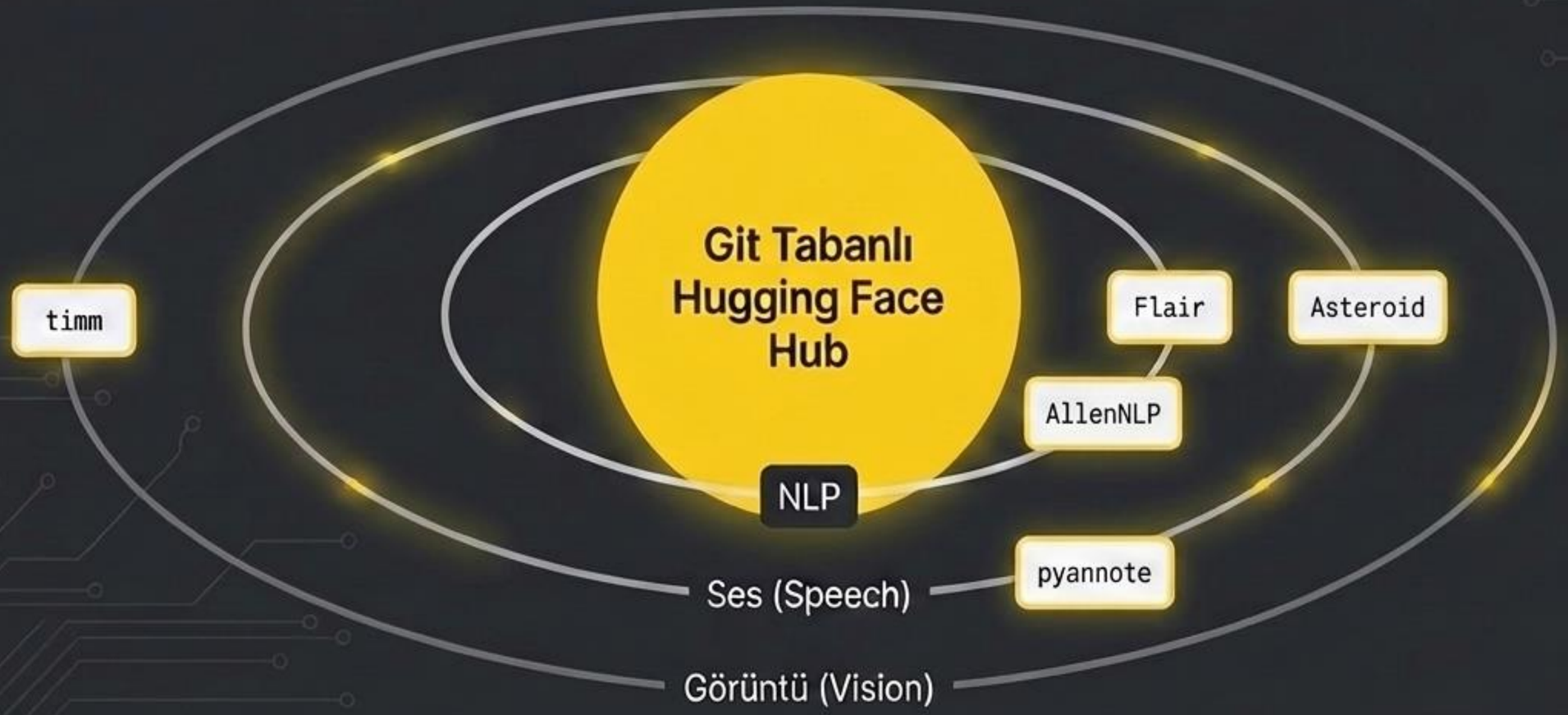
02. **Entegre Paylaşım:**
Trainer API ve push_to_hub().

03. **Depo Kontrolü:**
huggingface_hub ve HTTP POST akışları.

04. **Dokümantasyon:**
Model kartları ve YAML meta verileri.

Sadece NLP Değil: Tüm Makine Öğrenimi İçin Merkezi Git Deposu

- Devasa Ölçek: 10.000'den fazla herkese açık (public) model.
- Platformdaki her model aslında versiyonlama ve tekrarlanabilirlik sağlayan bağımsız bir Git reposudur.



Hub Özellikleri: Barındırmadan Çıkarıma Kadar Tam Kapsamlı Altyapı

Git Sürüm Kontrolü ve Tekrarlanabilirlik

Modeller statik dosyalar değil, geçmişi izlenebilir versiyonlanmış depolardır.



Inference API ve Widget'lar

Modeli indirmeden doğrudan tarayıcı üzerinden, özel girdiler ve widget'larla test etme imkanı sunar.



Ücretsiz Ortam ve Özel (Private) Planlar

Açık kaynak modelleri paylaşmak tamamen ücretsizdir; şirket modelleri için özel planlar mevcuttur.



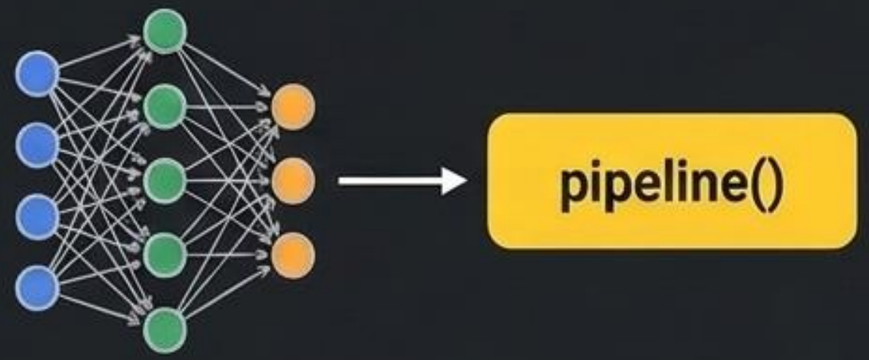
Hesap Açma Zorunluluğu

Depo yönetimi ve yükleme işlemleri için huggingface.co/join üzerinden bir kimlik gereklidir.



Önceden Eğitilmiş Modellerin Gücü: pipeline Entegrasyonu

- camembert-base modeli ve fill-mask görevi ile Fransızca metindeki maskeli kısım için olası kelimeler ve olasılık skorları elde edilir.
- **Kritik Kural:** Seçilen kontrol noktası (checkpoint) kullanılacağı göreve uygun olmalıdır; örneğin camembert-base modelinin başlığı (head) text-classification görevi için uygun değildir ve anlamsız sonuç üretir.
- Göreve uygun kontrol noktasını seçmek için Hugging Face Hub arayüzündeki Görev Seçici (Task Selector) kullanılmalıdır.




```
from transformers import pipeline

camembert_fill_mask = pipeline("fill-mask",
model="camembert-base")
results = camembert_fill_mask("Le camembert est
<mask> :)")
```

```
[
  {'sequence': 'Le camembert est délicieux :)', 'score':
0.49091, 'token': 7200, 'token_str': 'délicieux'},
  {'sequence': 'Le camembert est excellent :)', 'score':
0.10556, 'token': 2183, 'token_str': 'excellent'},
  {'sequence': 'Le camembert est succulent :)', 'score':
0.03453, 'token': 26202, 'token_str': 'succulent'},
  {'sequence': 'Le camembert est meilleur :)', 'score':
0.03303, 'token': 528, 'token_str': 'meilleur'},
  {'sequence': 'Le camembert est parfait :)', 'score':
0.03007, 'token': 1654, 'token_str': 'parfait'}
]
```

Mimari Sınıflar mı, Auto* Sınıfları mı?

Mimariye özgü sınıflar belirli bir mimariye bağlıdır. Auto* sınıfları ise model kontrol noktasına (checkpoint) göre uygun sınıfı seçerek farklı mimarilerle çalışmayı kolaylaştırır.

Mimariye Özgü Sınıflar

```
from transformers import CamembertTokenizer,
CamembertForMaskedLM

tokenizer = CamembertTokenizer.from_pretrained
 ("camembert-base")
model =
CamembertForMaskedLM.from_pretrained
("camembert-base")
```

Auto* Sınıfları (Mimariden Bağımsız - Önerilen)

```
from transformers import AutoTokenizer,
AutoModelForMaskedLM

tokenizer = AutoTokenizer.from_pretrained
("camembert-base")
model =
AutoModelForMaskedLM.from_pretrained
("camembert-base")
```

Yeni Model Deposu Oluşturmanın 3 Yolu

Yeni model deposu oluşturmanın üç ana yolu vardır: push_to_hub API, huggingface_hub Python kütüphanesi ve Web arayüzü.

1. push_to_hub API

2. huggingface_hub Kütüphanesi

3. Web Arayüzü

Ön Koşul: Paylaşım Öncesi Kimlik Doğrulama (Authentication)

Jupyter / Colab

```
from huggingface_hub import notebook_login
notebook_login()
```

Terminal / CLI

```
huggingface-cli login
```

Entegre Yükleme: Trainer API ile push_to_hub

Eğitim ile modeli paylaşma süreçlerini birleştirmenin en kolay yolu.

- TrainingArguments içindeki ilk parametre (“bert-finetuned-mrpc”), modelin hem yerel çıktı dizini'ni hem de Hub üzerindeki hedef reposunun adını belirler.
- push_to_hub=True parametresi, eğitim sırasında (save_strategy=“epoch” ile) modellerin otomatik olarak Hub'a senkronize edilmesini sağlar.
- Eğitim sonunda çağrılan trainer.push_to_hub() fonksiyonu son sürümü gönderir ve otomatik olarak hiperparametreleri ile değerlendirme sonuçlarını içeren bir Model Kartı (Model Card) oluşturur.

```python
from transformers import TrainingArguments, Trainer

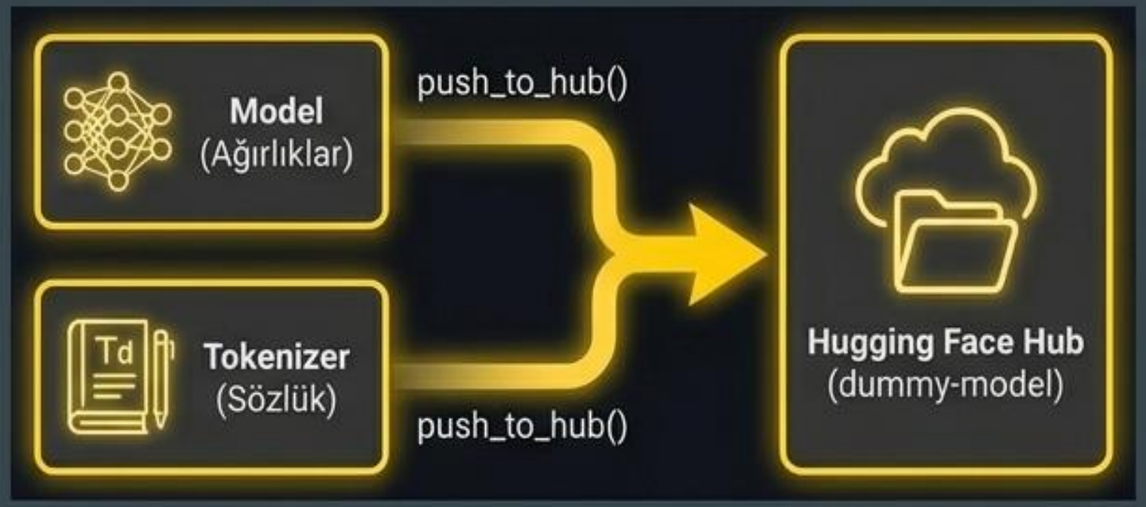
training_args = TrainingArguments(
    "bert-finetuned-mrpc",
    save_strategy="epoch",
    push_to_hub=True,
    hub_model_id="<namespace>/bert-finetuned-mrpc"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
)

trainer.train()
trainer.push_to_hub()
```

Doğrudan Yükleme: Model ve Tokenizer Seviyesinde push_to_hub()

- 1) Özelleştirilmiş döngülerde doğrudan model ve tokenizer nesnesinden yükleme.
- 2) organization parametresi ile takım alanlarına aktarım.
- 3) use_auth_token ile güvenli yetkilendirme.



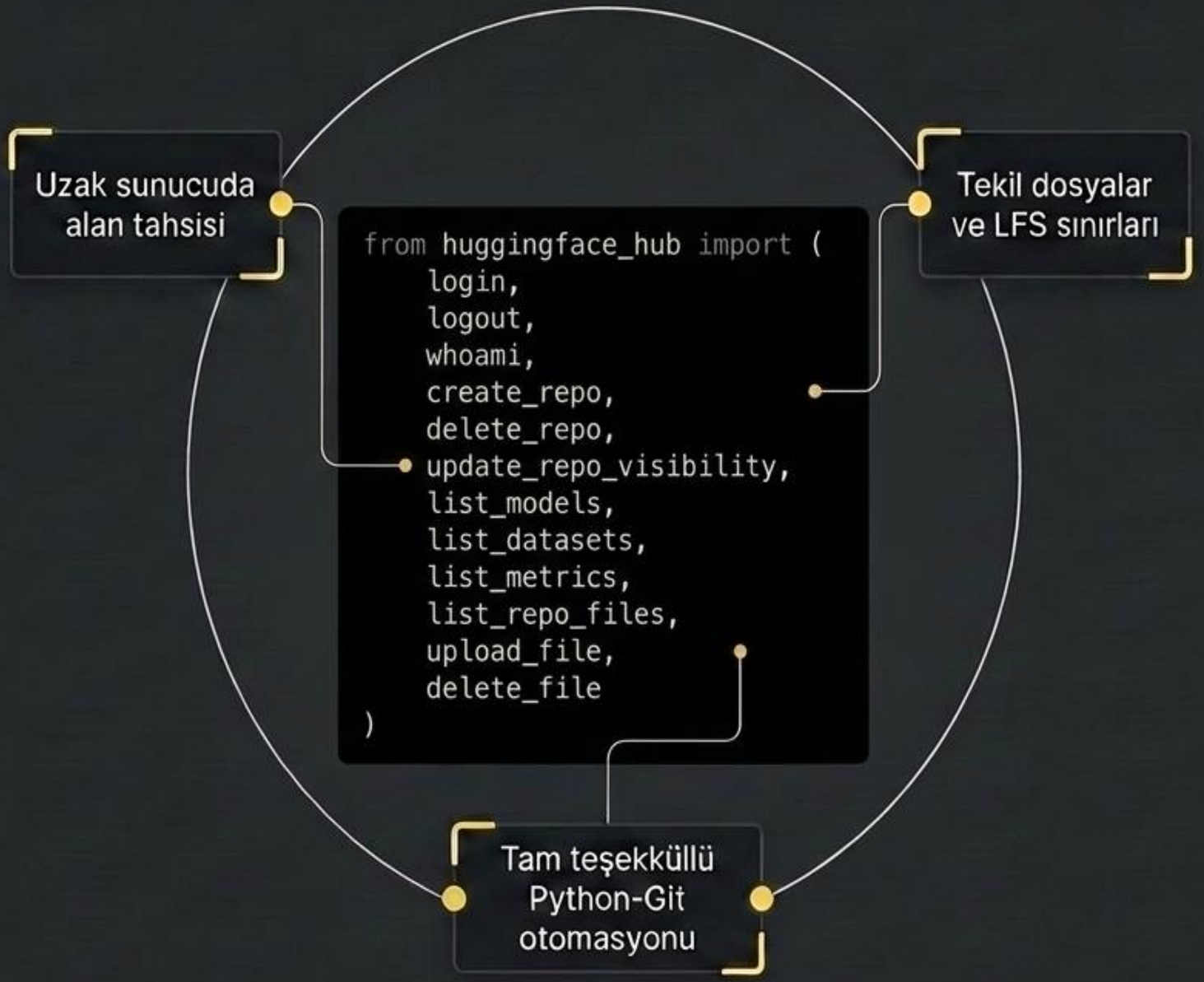

```python
from transformers import AutoModelForMaskedLM, AutoTokenizer

checkpoint = "camembert-base"
model = AutoModelForMaskedLM.from_pretrained(checkpoint)
tokenizer = AutoTokenizer.from_pretrained(checkpoint)

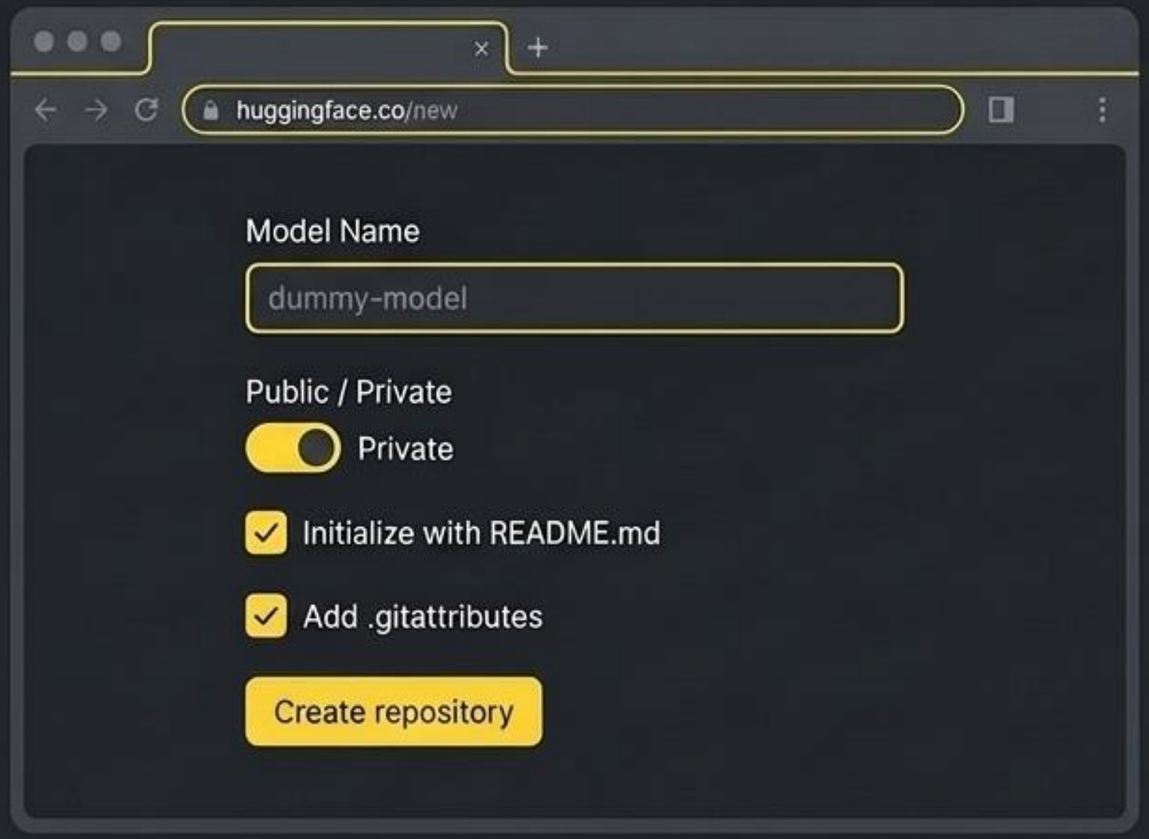
# Modeli kendi veya organizasyon alanınıza yükleyin
model.push_to_hub(
    "dummy-model",
    organization="huggingface",
    use_auth_token="<TOKEN>"
)
tokenizer.push_to_hub(
    "dummy-model",
    organization="huggingface",
    use_auth_token="<TOKEN>"
)
```

Kaputun Altındaki Motor: huggingface_hub Kütüphanesi

transformers kütüphanesinin de arka planda kullandığı temel araç seti. Dosya tabanlı yönetim ve Git etkileşimi için tam kontrol.



Depo (Repository) Oluşturma: Web Arayüzü vs. API create_repo





Tekil Dosya Yükleme ve Hub'a Doğrudan İletim (HTTP POST)

upload_file yaklaşımı sistemde git ve git-lfs kurulu olmasını gerektirmez; dosyaları HTTP POST istekleriyle doğrudan Hub'a iter. (Not: Boyutu 5GB'den büyük olan dosyalar bu yöntemle işlenemez).

```python
from huggingface_hub import upload_file

upload_file(
    "<path_to_file>/config.json",
    path_in_repo="config.json",
    repo_id="<namespace>/dummy-model",
)
```



Uçtan Uca Python ile Git Akışı: Repository Sınıfı

repository_flow.py

```python
from huggingface_hub import Repository

# 1. Uzak repoyu lokal klasöre klonlama
repo = Repository("<path_to_dummy_folder>", clone_from="<namespace>/dummy-model")

# 2. En güncel hali çekme
repo.git_pull()

# 3. Model ve Tokenizer dosyalarını lokal klasöre kaydetme
model.save_pretrained("<path_to_dummy_folder>")
tokenizer.save_pretrained("<path_to_dummy_folder>")

# 4. Dosyaları ekleme, commit ve push
repo.git_add()
repo.git_commit("Add model and tokenizer files")
repo.git_push()
```

git clone

git pull

git add & commit & push

Raw Git Yaklaşımı (Bölüm 1): Kurulum ve Klonlama

Python API'sini tamamen bypass edip saf Git kullanmak.

Terminal

```
$ git lfs install
Updated git hooks.
Git LFS initialized.

$ git clone https://huggingface.co/<namespace>/dummy-model
Cloning into 'dummy-model'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0)
Unpacking objects: 100% (3/3), done.

$ cd dummy-model && ls
README.md
```

```
model.save_pretrained(".")
tokenizer.save_pretrained(".")
```

Raw Git Yaklaşımı (Bölüm 2): LFS Üzerinden Büyük Dosyaları İtmek

Terminal

```
$ ls -al
total 432540
drwxr-xr-x 2 user user      4096 Dec 14 10:30 .
drwxr-xr-x 3 user user      4096 Dec 14 10:30 ..
-rw-r--r-- 1 user user       516 Dec 14 10:30 config.json
-rw-r--r-- 1 user user 436302525 Dec 14 10:30 pytorch_model.bin
-rw-r--r-- 1 user user       816 Dec 14 10:30 README.md
-rw-r--r-- 1 user user    810487 Dec 14 10:30 sentencepiece.bpe.model
-rw-r--r-- 1 user user       156 Dec 14 10:30 special_tokens_map.json
-rw-r--r-- 1 user user   2051184 Dec 14 10:30 tokenizer.json
-rw-r--r-- 1 user user       393 Dec 14 10:30 tokenizer_config.json

$ git add .
$ git status

      new file:   config.json
      new file:   pytorch_model.bin
      new file:   sentencepiece.bpe.model
      new file:   tokenizer.json

$ git lfs status
pytorch_model.bin (LFS)
sentencepiece.bpe.model (LFS)

$ git commit -m "First model version"
$ git push
```

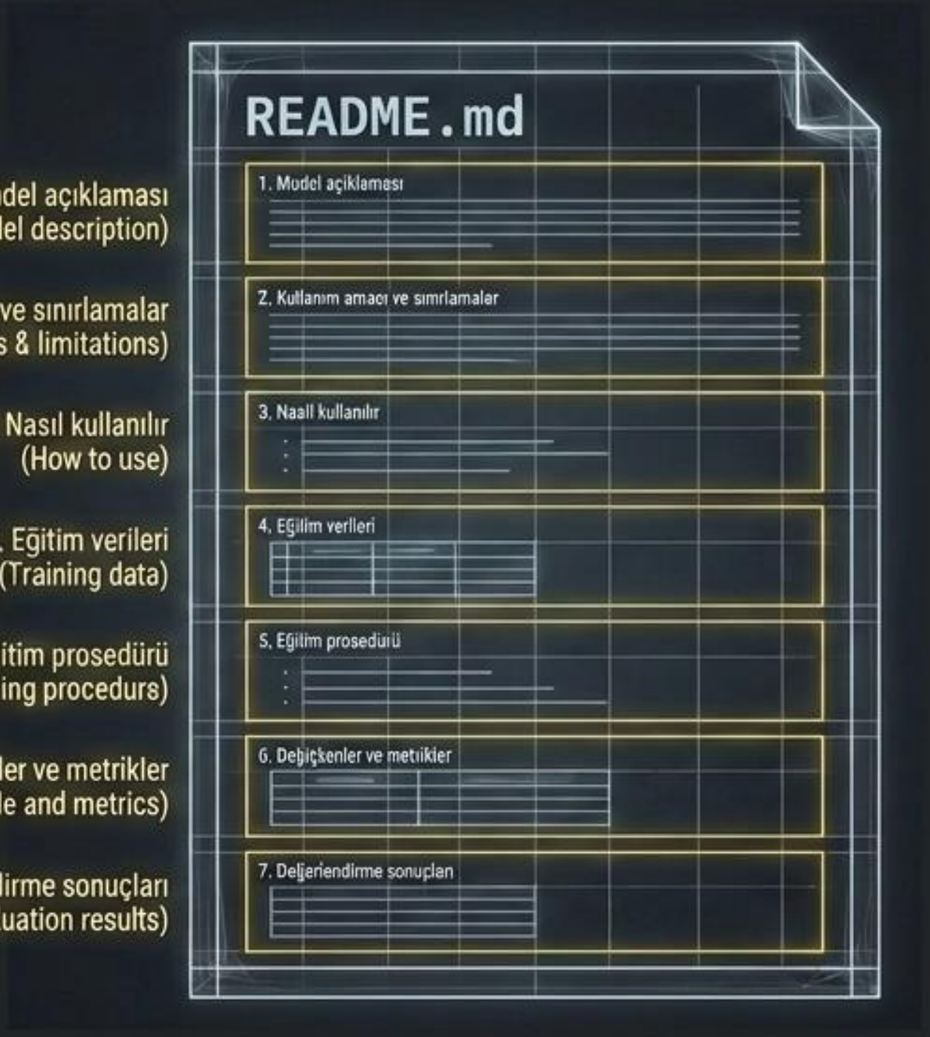


Web UI Diff

Model Kartı Oluşturma: Dokümantasyon, Yeniden Kullanılabilirlik ve Tekrarlanabilirlik

Model kartı (README.md), bir modelin yeniden kullanılabilmesini (reusability) ve sonuçlarının tekrarlanabilirliğini (reproducibility) sağlar.

Eğitim sürecini, kullanım bağlamını, sınırlamaları ve önyargıları (biases) belgeler (Temel referans: Margaret Mitchell vd., 2018).



Model Kartının Anatomisi (Bölüm 1): Temel Tanımlar



Model description (Model Açıklaması)

Modelin ne olduğu, mimarisi, sürümü, yazarı/kurumu, varsa ilgili akademik makalesi ve lisans bilgileri hakkında temel özet.

Intended uses & limitations (Kullanım Amacı ve Sınırlamalar)

Modelin amaçlanan kullanım alanları (desteklediği görevler ve diller), modelin uygun olmadığı veya hatalı/önyargılı (biased) sonuç verebileceği durumlar.

How to use (Nasıl Kullanılır)

Modelin nasıl kullanılacağını gösteren pratik örnekler; pipeline() fonksiyonu veya doğrudan model ve tokenizer sınıflarıyla çalışma kodları.

Model Kartının Anatomisi (Bölüm 2): Eğitim ve Değerlendirme Çıktıları

Training data
Eğitim, doğrulama ve test setlerinin dağılımı ve kökenleri.

Training procedure
Batch size, learning rate, epoch sayısı, donanım türü (GPU/TPU).

Variable and metrics
Optimize edilen hedefler (Accuracy, F1-score) ve kontrol değişkenleri.

Evaluation results
İstatistiksel sonuçlar ve test seti üzerindeki performans çıktısı. Karar eşiği (decision threshold) gibi parametreler içerir.



Metric	Value	Dataset
Accuracy	92.5%	Test Set
F1-score	0.901	Test Set
Precision	0.915	Test Set
Recall	0.889	Test Set

Not: Sağdaki grafik ve tablo temsili verilerdir, orijinal eğitimden

Arama Motorunu Beslemek: YAML Meta Verileri

Model kartının (README.md) en üstünde yer alan YAML bloğu, Hugging Face Hub'ın arama filtrelerini ve Inference widget'larını otomatik olarak besler.

```yaml
---
language: fr
license: mit
datasets:
  - oscar
---
```



İlham Alınabilecek Endüstri Standardı Model Kartları:

bert-base-cased | gpt2 | distilbert

Bölüm Özeti: Tüketimden Katkıya Tam Döngü

Hugging Face ekosisteminde modelleri kullanmaktan açık kaynak topluluğuna güvenilir modeller kazandırmaya uzanan 4 temel adım:

- **Model Keşfi ve Kullanımı:** pipeline() fonksiyonu ile hızlı çıkarım ve mimariden bağımsız Auto* (AutoTokenizer, AutoModel*) sınıflarıyla esnek entegrasyon.
- **Doğrudan ve Entegre Paylaşım:** Eğitim sürecinde Trainer API (push_to_hub=True) veya nesne seviyesinde doğrudan model.push_to_hub() / tokenizer.push_to_hub() kullanımı.
- **Düşük Seviye Depo Kontrolü:** huggingface_hub kütüphanesi (create_repo, Repository), Git gerektirmeyen upload_file (HTTP POST) ve büyük ağırlıklar için Git-LFS iş akışı.
- **Model Kartı ve Dokümantasyon:** Yeniden kullanılabilirlik, tekrarlanabilirlik ve sınırlamaların/önyargıların şeffafça belgelenmesi için README.md ve YAML meta verileri.